

LogisticRegression#

[https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.h](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html)

Description:

Logistic Regression (aka logit, MaxEnt) classifier. This class implements regularized logistic regression using the 'liblinear' library, 'newton-cg', 'sag', 'saga' and 'lbfgs' solvers. Note that regularization is applied by default. It can handle both dense and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied). The 'newton-cg', 'sag', and 'lbfgs' solvers support only L2 regularization with primal formulation, or no regularization. The 'liblinear' solver supports both L1 and L2 regularization, with a dual formulation only for the L2 penalty. The Elastic-Net regularization is only supported by the 'saga' solver. For multiclass problems, all solvers but 'liblinear' optimize the (penalized) multinomial loss. 'liblinear' only handle binary classification but can be extended to handle multiclass by using OneVsRestClassifier.

Function Signature

```
class sklearn.linear_model.LogisticRegression(penalty='l2',
*, dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None,
solver='lbfgs', max_iter=100, multi_class='deprecated',
verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
[source]#
```

Parameters:

penalty{'l1', 'l2', 'elasticnet', None}, default='l2'

dualbool, default=False

tolfloat, default=1e-4

```
class sklearn.linear_model.LogisticRegression(penalty='l2',
*, dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None,
solver='lbfgs', max_iter=100, multi_class='deprecated',
verbose=0, warm_start=False, n_jobs=None, l1_ratio=None)
[source]#
```

Parameters:

penalty{'l1', 'l2', 'elasticnet', None}, default='l2'

dualbool, default=False

tolfloat, default=1e-4

decision_function(X)[source]#

Parameters:

X{array-like, sparse matrix} of shape (n_samples, n_features)

Returns:

scoresndarray of shape (n_samples,) or (n_samples, n_classes)

Parameters

`tolfloat, default=1e`

Tolerance for stopping criteria.

`solver{'lbfgs', 'liblinear', 'newton`

Algorithm to use in the optimization problem. Default is 'lbfgs'. To choose a solver, you might want to consider the following aspects: For small datasets, 'liblinear' is a good choice, whereas 'sag' and 'saga' are faster for large ones; For multiclass problems, all solvers except 'liblinear' minimize the full multinomial loss; 'liblinear' can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the `OneVsRestClassifier`. 'newton-cholesky' is a good choice for $n_{\text{samples}} \gg n_{\text{features}} * n_{\text{classes}}$, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on $n_{\text{features}} * n_{\text{classes}}$ because it explicitly computes the full Hessian ma

`X{array`

The data matrix for which we want to get the confidence scores.

`X{array}`

Training vector, where `n_samples` is the number of samples and `n_features` is the number of features.

`yarray`

Target vector relative to `X`.

`sample_weightarray`

Array of weights that are assigned to individual samples. If not provided, then each sample is given unit weight. Added in version 0.17: `sample_weight` support to `LogisticRegression`.

`paramsdict`

Parameter names mapped to their values.

`X{array}`

The data matrix for which we want to get the predictions.

Xarray

Vector to be scored, where `n_samples` is the number of samples and `n_features` is the number of features.

Tarray

Returns the log-probability of the sample for each class in the model, where classes are ordered as they are in `self.classes_`.

Xarray

Vector to be scored, where `n_samples` is the number of samples and `n_features` is the number of features.

Tarray

Returns the probability of the sample for each class in the model, where classes are ordered as they are in `self.classes_`.

Xarray

Test samples.

yarray

True labels for X.

sample_weightarray

Sample weights.

****paramsdict**

Estimator parameters.

Code Examples

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.82e-01, 1.82e-02, 1.44e-08],
       [9.72e-01, 2.82e-02, 3.02e-08]])
>>> clf.score(X, y)
0.97
```

Notes & Warnings

WARNING

Warning Some penalties may not work with some solvers. See the parameter solver below, to know the compatibility between the penalty and solver.

NOTE

Note The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) intercept_scaling has to be increased.

WARNING

Warning The choice of the algorithm depends on the penalty chosen and on (multinomial) multiclass support:

solver	penalty	multinomial	multiclass
'lbfgs'	'l2', None	yes	
'liblinear'	'l1', 'l2'	no	
'newton-cg'	'l2', None	yes	
'newton-cholesky'	'l2', None	yes	
'sag'	'l2', None	yes	
'saga'	'elasticnet', 'l1', 'l2', None	yes	

NOTE

Note 'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from sklearn.preprocessing.

NOTE

See also Refer to the User Guide for more information regarding LogisticRegression and more specifically the Table summarizing solver/penalty supports.

NOTE

See also SGDClassifierIncrementally trained logistic regression (when given the parameter `loss="log_loss"`). LogisticRegressionCVLogistic regression with built-in cross validation.

Detailed Documentation

Documentation

Logistic Regression (aka logit, MaxEnt) classifier.

This class implements regularized logistic regression using the ‘liblinear’ library, ‘newton-cg’, ‘sag’, ‘saga’ and ‘lbfgs’ solvers. Note that regularization is applied by default. It can handle both dense and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied).

The ‘newton-cg’, ‘sag’, and ‘lbfgs’ solvers support only L2 regularization with primal formulation, or no regularization. The ‘liblinear’ solver supports both L1 and L2 regularization, with a dual formulation only for the L2 penalty. The Elastic-Net regularization is only supported by the ‘saga’ solver.

For multiclass problems, all solvers but ‘liblinear’ optimize the (penalized) multinomial

loss. 'liblinear' only handle binary classification but can be extended to handle multiclass by using `OneVsRestClassifier`.

Read more in the User Guide.

Specify the norm of the penalty:

None: no penalty is added;

'l2': add a L2 penalty term and it is the default choice;

'l1': add a L1 penalty term;

'elasticnet': both L1 and L2 penalty terms are added.

Some penalties may not work with some solvers. See the parameter `solver` below, to know the compatibility between the penalty and solver.

Added in version 0.19: l1 penalty with SAGA solver (allowing 'multinomial' + L1)

Dual (constrained) or primal (regularized, see also this equation) formulation. Dual formulation is only implemented for l2 penalty with liblinear solver. Prefer `dual=False` when `n_samples > n_features`.

Tolerance for stopping criteria.

Inverse of regularization strength; must be a positive float. Like in support vector machines, smaller values specify stronger regularization.

Specifies if a constant (a.k.a. bias or intercept) should be added to the decision function.

Useful only when the solver liblinear is used and `self.fit_intercept` is set to `True`. In this case, `x` becomes `[x, self.intercept_scaling]`, i.e. a "synthetic" feature with constant value

equal to `intercept_scaling` is appended to the instance vector. The intercept becomes `intercept_scaling * synthetic_feature_weight`.

The synthetic feature weight is subject to L1 or L2 regularization as all other features. To lessen the effect of regularization on synthetic feature weight (and therefore on the intercept) `intercept_scaling` has to be increased.

Weights associated with classes in the form `{class_label: weight}`. If not given, all classes are supposed to have weight one.

The “balanced” mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

Note that these weights will be multiplied with `sample_weight` (passed through the fit method) if `sample_weight` is specified.

Added in version 0.17: `class_weight='balanced'`

Used when `solver == 'sag', 'saga' or 'liblinear'` to shuffle the data. See Glossary for details.

Algorithm to use in the optimization problem. Default is `'lbfgs'`. To choose a solver, you might want to consider the following aspects:

For small datasets, `'liblinear'` is a good choice, whereas `'sag'` and `'saga'` are faster for large ones;

For multiclass problems, all solvers except `'liblinear'` minimize the full multinomial loss;

`'liblinear'` can only handle binary classification by default. To apply a one-versus-rest scheme for the multiclass setting one can wrap it with the `OneVsRestClassifier`.

'newton-cholesky' is a good choice for $n_samples \gg n_features * n_classes$, especially with one-hot encoded categorical features with rare categories. Be aware that the memory usage of this solver has a quadratic dependency on $n_features * n_classes$ because it explicitly computes the full Hessian matrix.

The choice of the algorithm depends on the penalty chosen and on (multinomial) multiclass support:

multinomial multiclass

'elasticnet', 'l1', 'l2', None

'sag' and 'saga' fast convergence is only guaranteed on features with approximately the same scale. You can preprocess the data with a scaler from `sklearn.preprocessing`.

Refer to the User Guide for more information regarding LogisticRegression and more specifically the Table summarizing solver/penalty supports.

Added in version 0.17: Stochastic Average Gradient (SAG) descent solver. Multinomial support in version 0.18.

Added in version 0.19: SAGA solver.

Changed in version 0.22: The default solver changed from 'liblinear' to 'lbfgs' in 0.22.

Added in version 1.2: newton-cholesky solver. Multinomial support in version 1.6.

Maximum number of iterations taken for the solvers to converge.

If the option chosen is 'ovr', then a binary problem is fit for each label. For 'multinomial' the loss minimised is the multinomial loss fit across the entire probability distribution, even when the data is binary. 'multinomial' is unavailable when `solver='liblinear'`. 'auto' selects 'ovr' if the data is binary, or if `solver='liblinear'`, and otherwise selects

'multinomial'.

Added in version 0.18: Stochastic Average Gradient descent solver for 'multinomial' case.

Changed in version 0.22: Default changed from 'ovr' to 'auto' in 0.22.

Deprecated since version 1.5: `multi_class` was deprecated in version 1.5 and will be removed in 1.8. From then on, the recommended 'multinomial' will always be used for `n_classes >= 3`. Solvers that do not support 'multinomial' will raise an error. Use `sklearn.multiclass.OneVsRestClassifier(LogisticRegression())` if you still want to use OvR.

For the liblinear and lbfgs solvers set `verbose` to any positive number for verbosity.

When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. Useless for liblinear solver. See the Glossary.

Added in version 0.17: `warm_start` to support lbfgs, newton-cg, sag, saga solvers.

Number of CPU cores used when parallelizing over classes if `multi_class='ovr'`. This parameter is ignored when the solver is set to 'liblinear' regardless of whether 'multi_class' is specified or not. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See Glossary for more details.

The Elastic-Net mixing parameter, with $0 \leq l1_ratio \leq 1$. Only used if `penalty='elasticnet'`. Setting `l1_ratio=0` is equivalent to using `penalty='l2'`, while setting `l1_ratio=1` is equivalent to using `penalty='l1'`. For $0 < l1_ratio < 1$, the penalty is a combination of L1 and L2.

A list of class labels known to the classifier.

Coefficient of the features in the decision function.

`coef_` is of shape $(1, n_features)$ when the given problem is binary. In particular, when `multi_class='multinomial'`, `coef_` corresponds to outcome 1 (True) and `-coef_` corresponds to outcome 0 (False).

Intercept (a.k.a. bias) added to the decision function.

If `fit_intercept` is set to False, the intercept is set to zero. `intercept_` is of shape $(1,)$ when the given problem is binary. In particular, when `multi_class='multinomial'`, `intercept_` corresponds to outcome 1 (True) and `-intercept_` corresponds to outcome 0 (False).

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Actual number of iterations for all classes. If binary or multinomial, it returns only 1 element. For liblinear solver, only the maximum number of iteration across all classes is given.

Changed in version 0.20: In SciPy $\leq 1.0.0$ the number of lbfgs iterations may exceed `max_iter`. `n_iter_` will now report at most `max_iter`.

Incrementally trained logistic regression (when given the parameter `loss="log_loss"`).

Logistic regression with built-in cross validation.

The underlying C implementation uses a random number generator to select features

when fitting the model. It is thus not uncommon, to have slightly different results for the same input data. If that happens, try with a smaller tol parameter.

Predict output may not match that of standalone liblinear in certain cases. See differences from liblinear in the narrative documentation.

Ciyou Zhu, Richard Byrd, Jorge Nocedal and Jose Luis Morales.
<http://users.iems.northwestern.edu/~nocedal/lbfgsb.html>

<https://www.csie.ntu.edu.tw/~cjlin/liblinear/>

Minimizing Finite Sums with the Stochastic Average Gradient <https://hal.inria.fr/hal-00860051/document>

“SAGA: A Fast Incremental Gradient Method With Support for Non-Strongly Convex Composite Objectives”

methods for logistic regression and maximum entropy models. Machine Learning 85(1-2):41-75. https://www.csie.ntu.edu.tw/~cjlin/papers/maxent_dual.pdf

For a comparison of the LogisticRegression with other classifiers see: Plot classification probability.

Predict confidence scores for samples.

The confidence score for a sample is proportional to the signed distance of that sample to the hyperplane.

The data matrix for which we want to get the confidence scores.

Confidence scores per (n_samples, n_classes) combination. In the binary case, confidence score for self.classes_[1] where >0 means this class would be predicted.

Convert coefficient matrix to dense array format.

Converts the `coef_` member (back) to a `numpy.ndarray`. This is the default format of `coef_` and is required for fitting, so calling this method is only required on models that have previously been sparsified; otherwise, it is a no-op.

Fit the model according to the given training data.

Training vector, where `n_samples` is the number of samples and `n_features` is the number of features.

Target vector relative to `X`.

Array of weights that are assigned to individual samples. If not provided, then each sample is given unit weight.

Added in version 0.17: `sample_weight` support to `LogisticRegression`.

The SAGA solver supports both `float64` and `float32` bit arrays.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRequest` encapsulating routing information.

Get parameters for this estimator.

If `True`, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Predict class labels for samples in X.

The data matrix for which we want to get the predictions.

Vector containing the class labels for each sample.

Predict logarithm of probability estimates.

The returned estimates for all classes are ordered by the label of classes.

Vector to be scored, where `n_samples` is the number of samples and `n_features` is the number of features.

Returns the log-probability of the sample for each class in the model, where classes are ordered as they are in `self.classes_`.

Probability estimates.

The returned estimates for all classes are ordered by the label of classes.

For a multi_class problem, if `multi_class` is set to be “multinomial” the softmax function is used to find the predicted probability of each class. Else use a one-vs-rest approach, i.e. calculate the probability of each class assuming it to be positive using the logistic function and normalize these values across all the classes.

Vector to be scored, where `n_samples` is the number of samples and `n_features` is the number of features.

Returns the probability of the sample for each class in the model, where classes are ordered as they are in `self.classes_`.

Return accuracy on provided data and labels.

In multi-label classification, this is the subset accuracy which is a harsh metric since you require for each sample that each label set be correctly predicted.

Mean accuracy of `self.predict(X)` w.r.t. `y`.

Configure whether metadata should be requested to be passed to the fit method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options fo